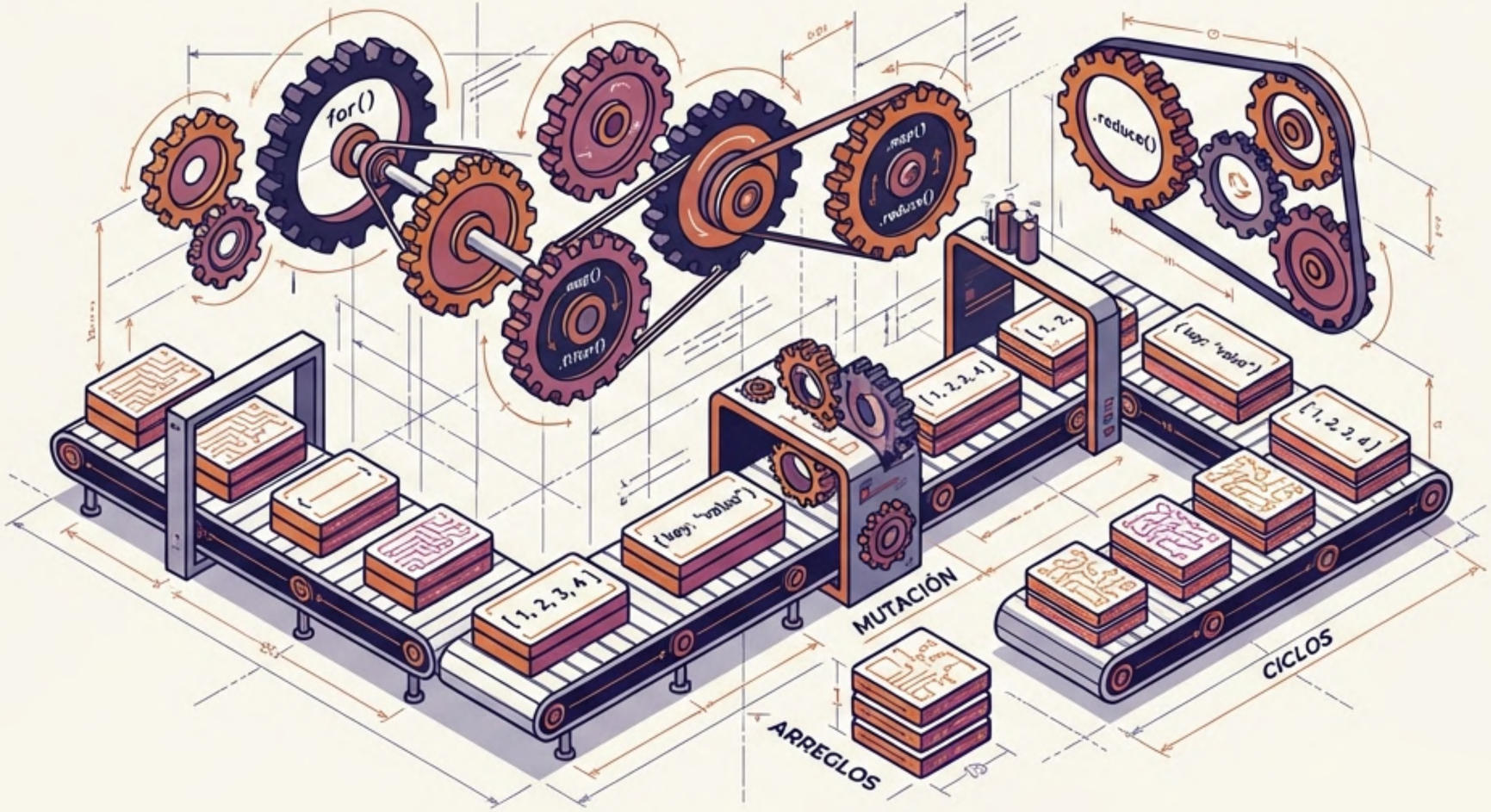
DOMINANDO LOS DATOS EN JAVASCRIPT

ARREGLOS, CICLOS Y MUTACIONES

Una guía visual definitiva para desarrolladores. De la memoria subyacente a la manipulación avanzada.



NotebookLM

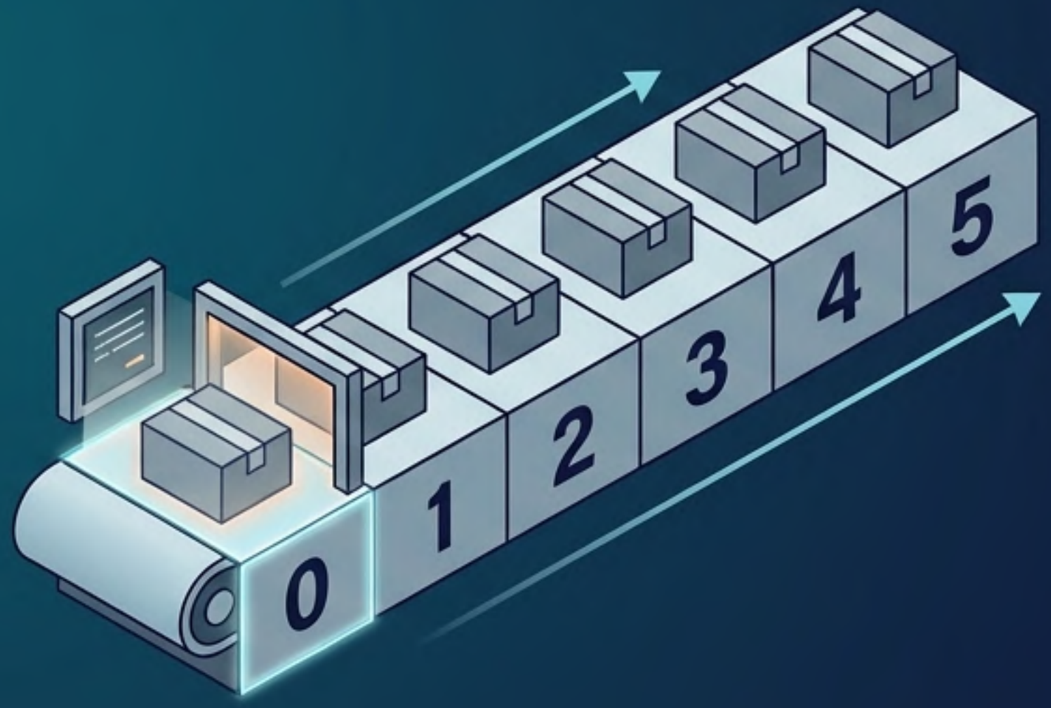
El Almacén de Datos: Objetos vs. Arreglos

Objetos

- Colecciones con llaves nombradas.
- Elementos desordenados.
- Ideales para mapear propiedades o diccionarios.
- Acceso: **obj.clave** o **obj['clave']**

Arreglos

- Colecciones ordenadas secuencialmente.
- Basados en índices numéricos (inician en 0).
- Ideales para listas y tamaños dinámicos.
- Acceso: **arr[0]**



NotebookLM

Anatomía de la Cinta Transportadora

Propiedad **.length = 3**

Índices (Zero-indexed)

El final del arreglo

Histórico: ~~arr[arr.length - 1]~~

Moderno: arr.at(-1)

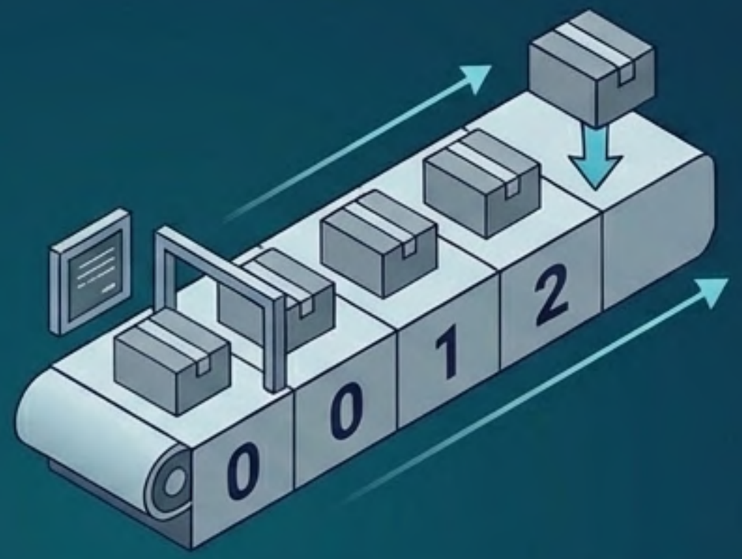
Soporta índices negativos para contar desde el final.

NotebookLM

Operaciones de Extremos y Rendimiento

El Final del Arreglo (Rápido)

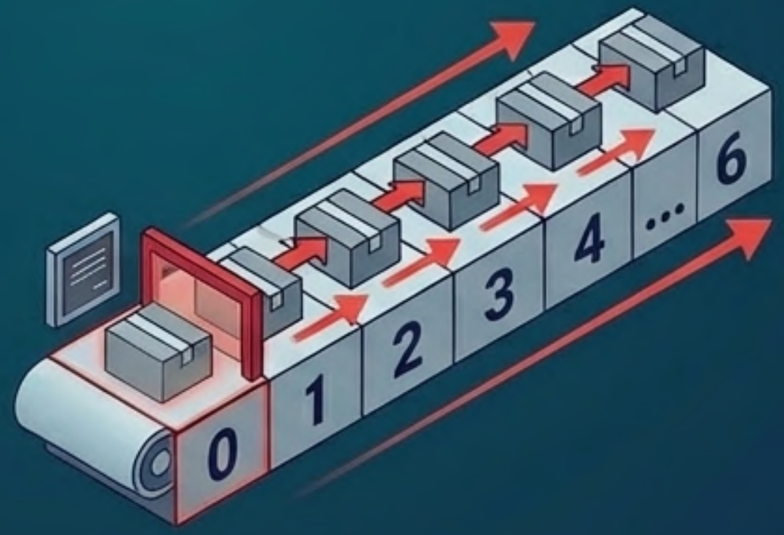
- push() inserta
- pop() extrae



O(1) - Instantáneo

El Inicio del Arreglo (Lento)

- unshift() inserta
- shift() extrae



O(N) - Re-indexación requerida

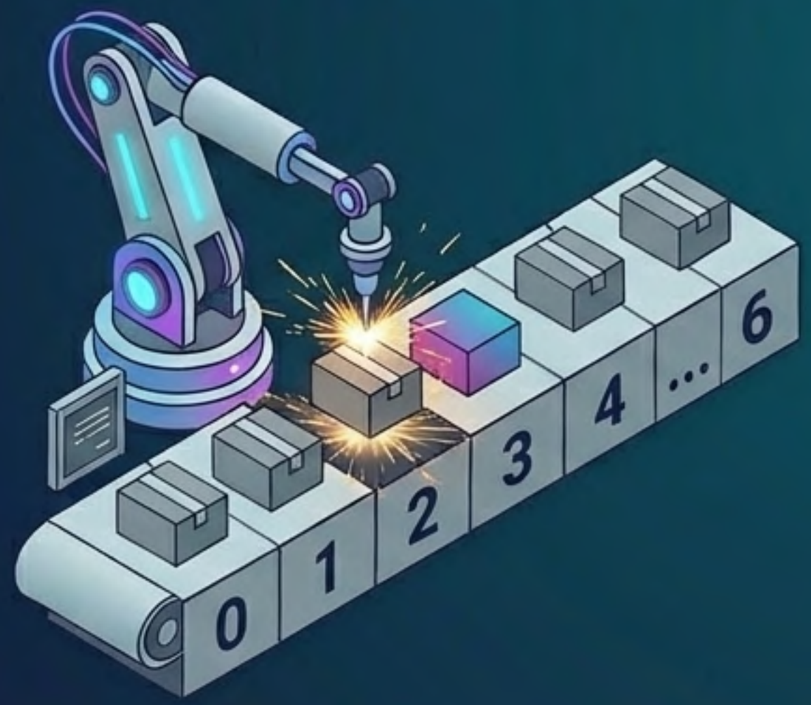
NotebookLM

Modificando el Medio: splice vs. slice

La Navaja Suiza: splice()

arr.splice(start, deleteCount, ...items)

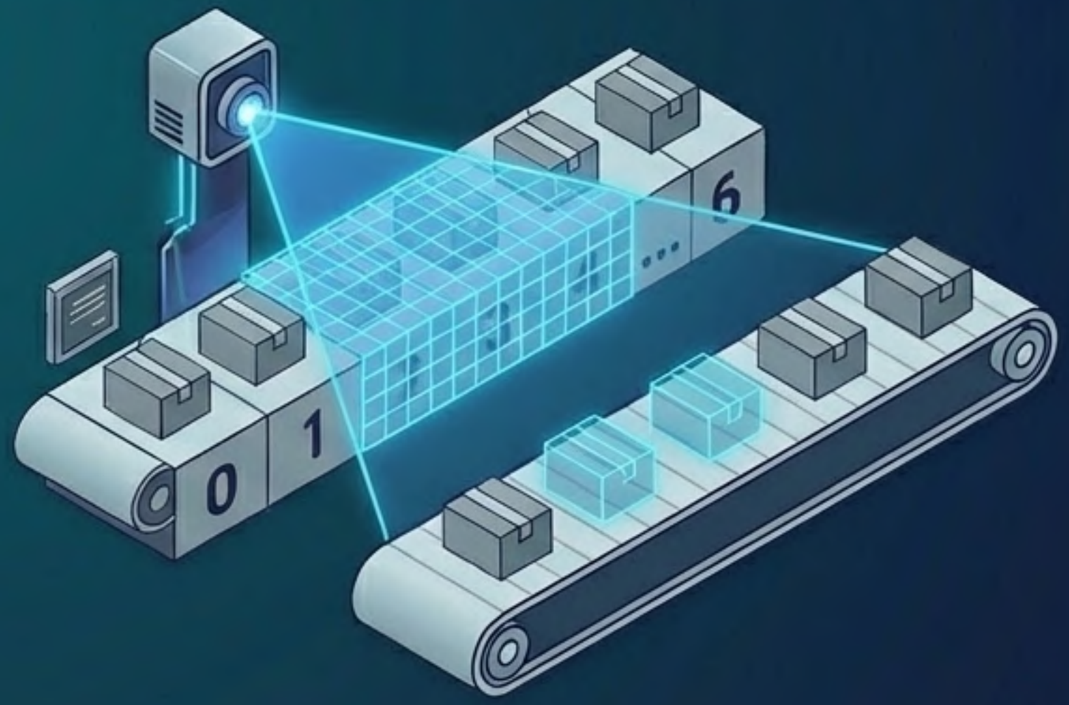
Destructivo: Modifica el arreglo original directamente. Puede eliminar, reemplazar o insertar elementos.



La Fotocopiadora: slice()

arr.slice(start, end)

No Destructivo: No toca el arreglo original. Devuelve una copia superficial de la porción solicitada.



NotebookLM

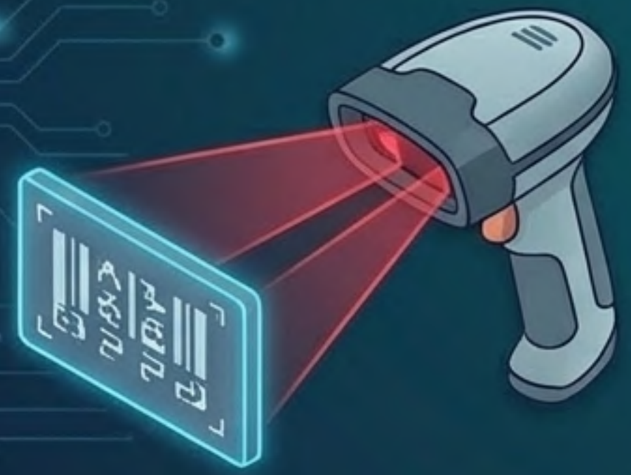
Radares de Búsqueda

Búsqueda Exacta

indexOf(item) -> Devuelve el índice o –1

includes(item) -> Devuelve un booleano (true/false)

includes maneja correctamente el valor **NaN**. **indexOf** no.

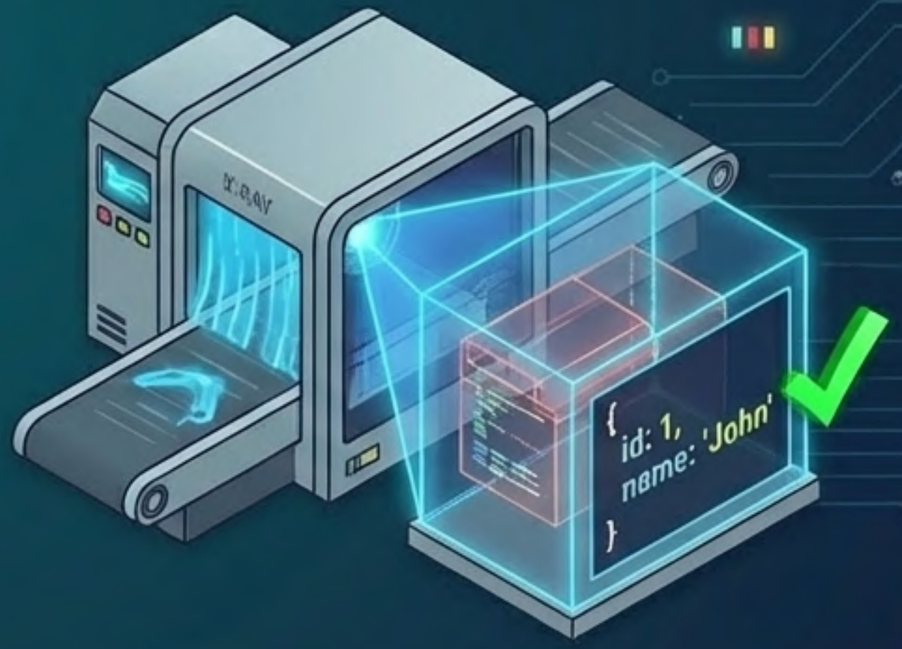


Búsqueda por Condición

find(fn) -> Devuelve el primer elemento que cumpla

findIndex(fn) -> Devuelve su posición

Ideal para encontrar objetos complejos usando lógica.



NotebookLM

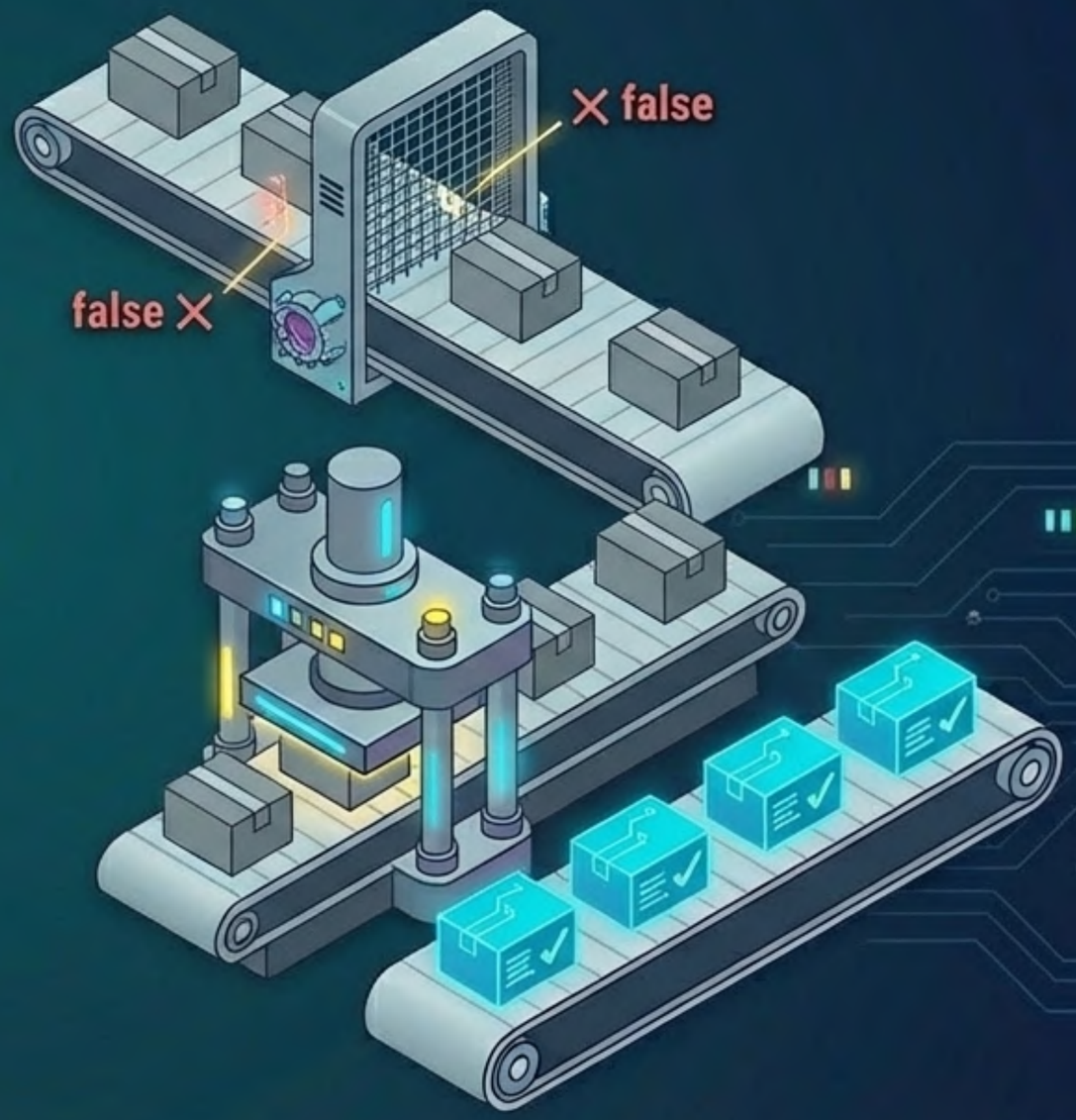
La Cadena de Montaje: Transformaciones No Destructivas

El Filtro de Malla: filter(fn)

Retorna un arreglo nuevo solo con los elementos donde la función devuelve true.

La Estampadora: map(fn)

Transforma cada elemento individualmente y retorna un arreglo del mismo tamaño.

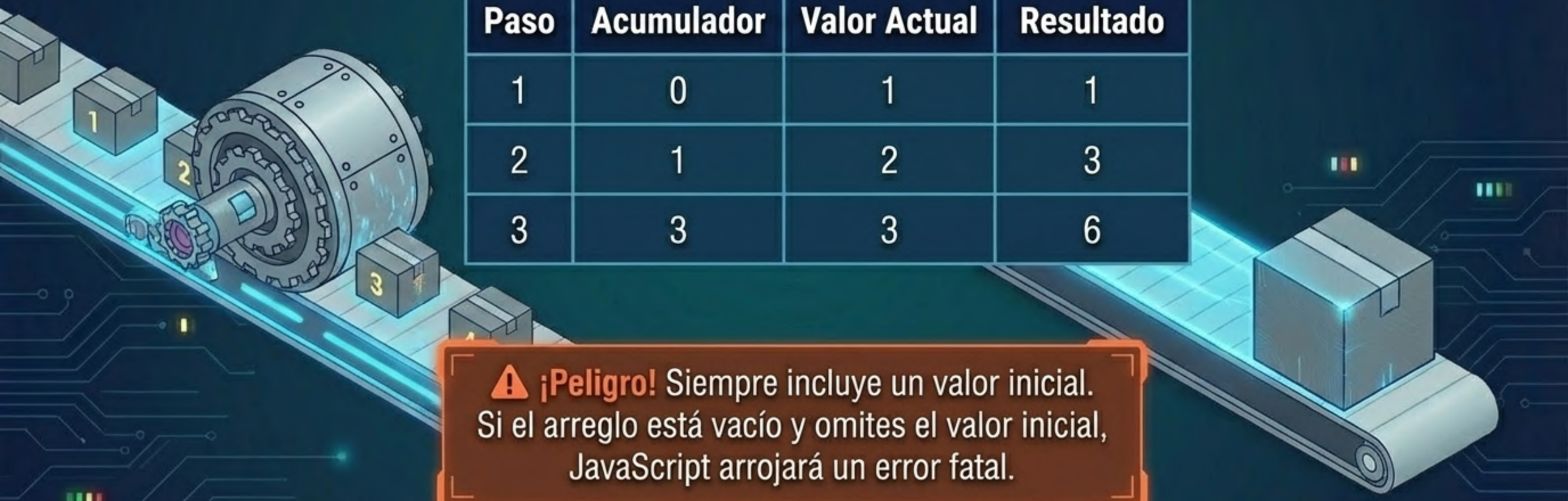


NotebookLM

El Compactador de Datos: reduce()

arr.reduce((acumulador, actual) => ..., inicial)

Paso	Acumulador	Valor Actual	Resultado
1	0	1	1
2	1	2	3
3	3	3	6



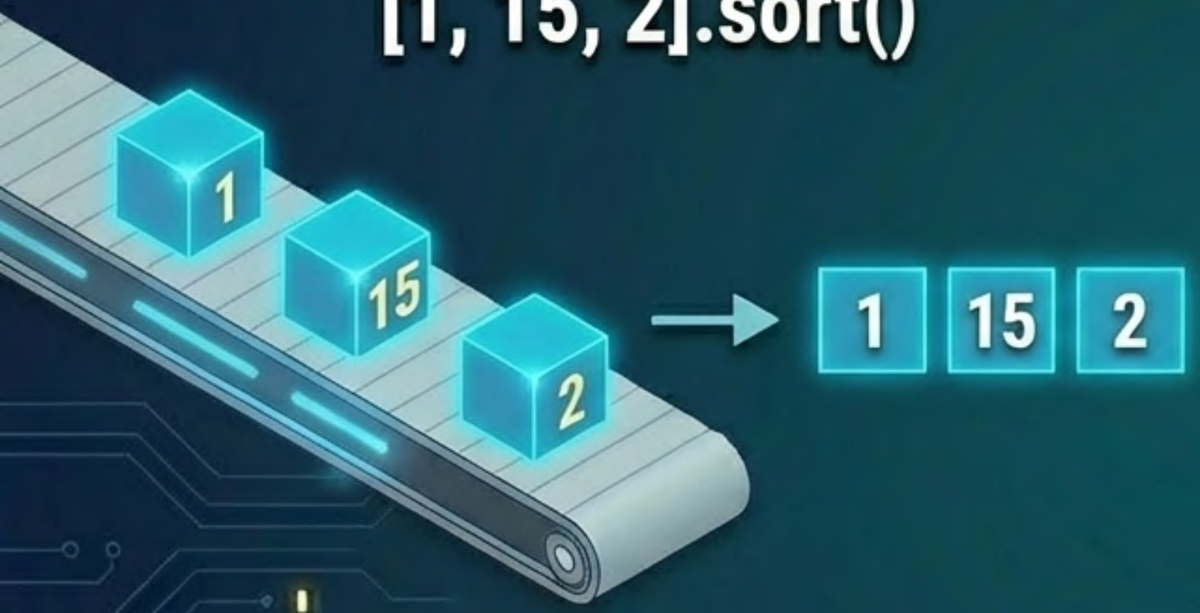
⚠ **¡Peligro!** Siempre incluye un valor inicial. Si el arreglo está vacío y omites el valor inicial, JavaScript arrojará un error fatal.

NotebookLM

El Peligro Oculto del Ordenamiento

El Comportamiento por Defecto

[1, 15, 2].sort()



Por defecto, JS convierte todo a texto y ordena alfabéticamente. ('2' es mayor que '15').

La Solución: Función de Comparación

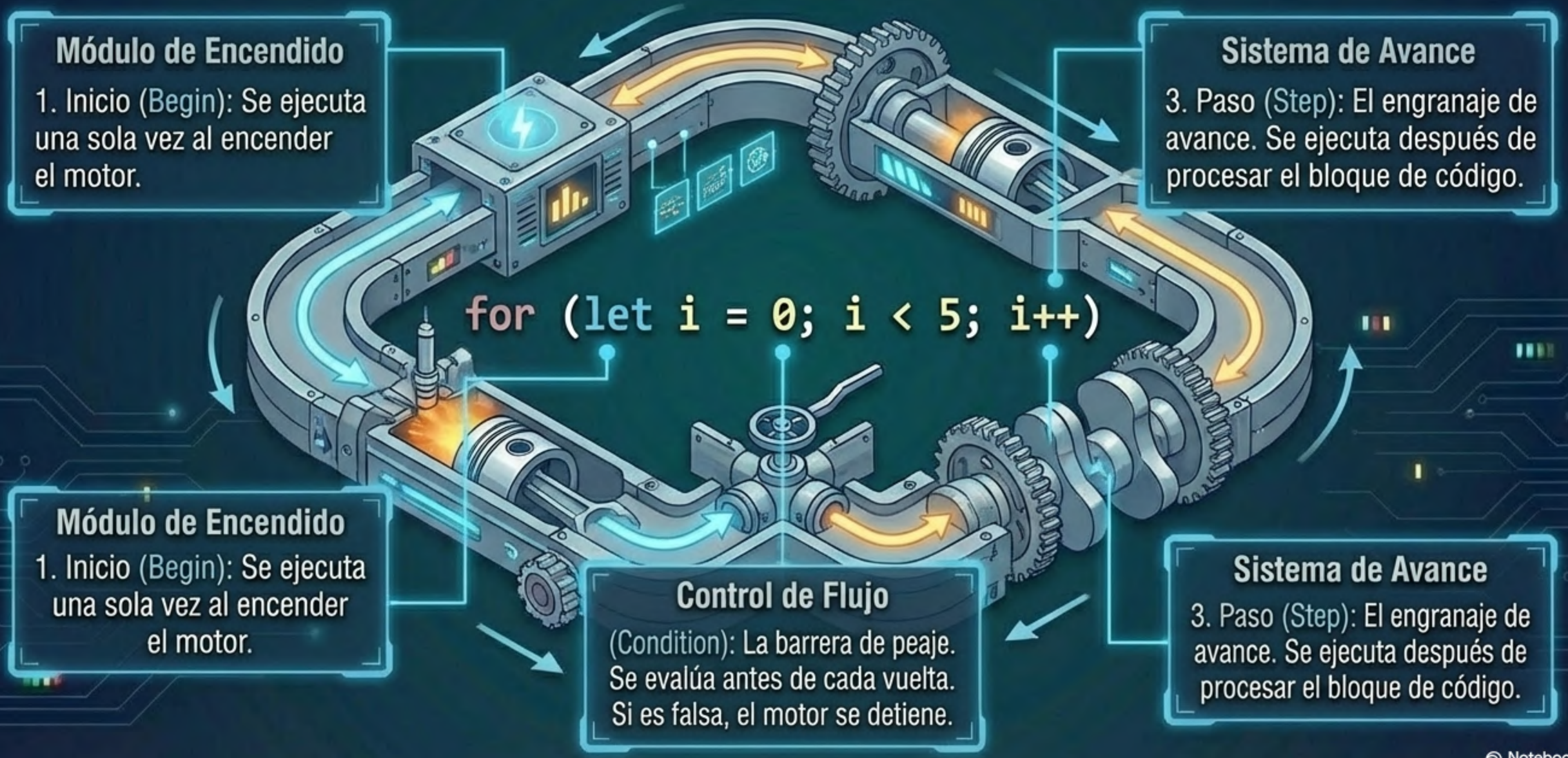
arr.sort((a, b) => a - b)

- ← - Retorna negativo: 'a' va primero
- → - Retorna positivo: 'b' va primero
- = - Retorna 0: se mantienen igual

Nota: sort() y reverse() son métodos destructivos.

NotebookLM

Motores de Navegación: Anatomía de un Ciclo



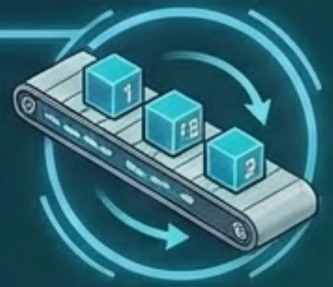
NotebookLM

Matriz de Decisión de Ciclos

¿Cuál ciclo debo usar?

for...of

Iteración directa sobre los valores de un arreglo (Sintaxis moderna).



for

Iteración clásica o matemática con control absoluto del índice.

while

Iteración basada en un evento externo o condición que cambiará eventualmente.

do...while

Garantizar que el código del ciclo se ejecute al menos una vez antes de verificar la condición.

⚠ Alerta de Rendimiento: Evita usar for...in con arreglos. Está diseñado para iterar sobre propiedades de objetos y es significativamente más lento.

NotebookLM

Control de Tráfico dentro del Ciclo

break (Salida de Emergencia)

Destruye y abandona el ciclo por completo.
El control pasa a la primera línea de código después del ciclo.

continue (El Desvío Rápido)

Omite el resto del código de la iteración actual.
Fuerza el inicio inmediato de la siguiente vuelta.

NotebookLM

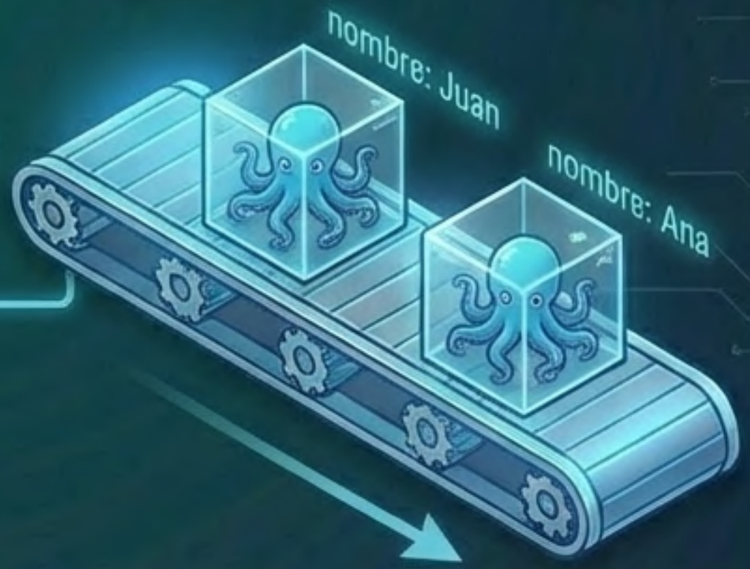
Cajas Mágicas: Datos Multidimensionales

Matrices (Arreglos de Arreglos)

[[1, 2], [3, 4]]

Conjuntos de Datos Reales
(Arreglos de Objetos)

[{ nombre: 'Juan' }, { nombre: 'Ana' }]



El arreglo proporciona la secuencia y el orden.
El objeto proporciona el significado y las etiquetas.

NotebookLM

Caso de Estudio: El Registro del Licántropo

Uniendo arreglos, objetos y ciclos para revelar patrones.

1. Almacenamiento

Arreglo de objetos diarios:

```
JOURNAL = [{
   events: ['maní'], squirrel: true },
}, ...]
```

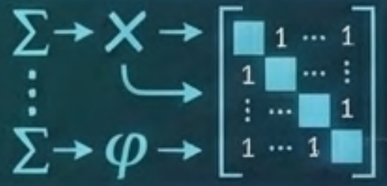
2. Navegación y Búsqueda

Un motor de ciclo iterando y filtrando:

```
for (let entry of JOURNAL) {
  if (entry.events.includes('maní')) ...
}
```

3. Síntesis Matemática

Mapear los resultados en una matriz estadística para calcular la correlación final (phi).



Insight: La combinación de estas tres estructuras permite a JavaScript modelar y analizar eventos del mundo real.

NotebookLM

La Consola Central (Cheat Sheet)

Mutadores (Destructivos)

- push(), pop(), unshift(),
- shift(), splice(),
- sort(), reverse()

Accesores (Seguros)

- slice(),
- concat(),
- at()

Radares de Búsqueda

- Exacta:
  indexOf(), includes()
- Condicional:
  find(), findIndex()

Fábrica Funcional

map() → Transforma
filter() → Filtra
reduce() → Compacta

NotebookLM